# IE4497/EE6497

# *Pattern Recognition and Deep Learning*

**Weeks 8 – 13: Neural Networks & Deep Learning**

**Lecturer: Wang Lipo**

**Email: elpwang@ntu.edu.sg**

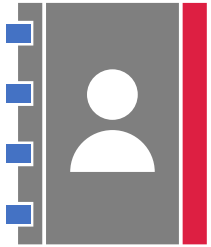
# Contact Information: Wang Lipo

School of EEE

Office: S1-B1C-98

Office Phone: 6916 3162 ext.3013

Email: elpwang@ntu.edu.sg



# Consulting

Email

Or make an appointment

Or just drop by at any time

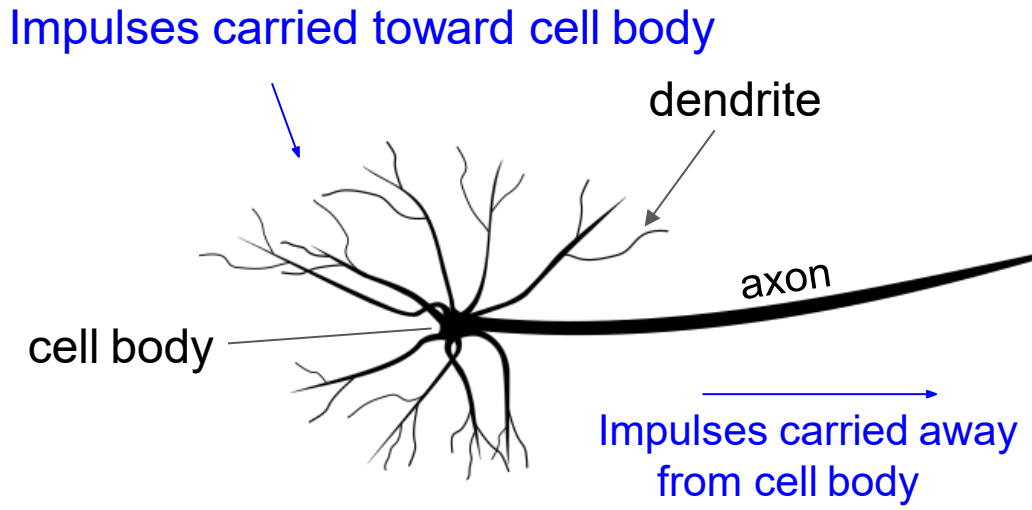
Your suggestions/comments are always welcome!



# Part 2.1: Introduction to Neural Networks and Deep Learning

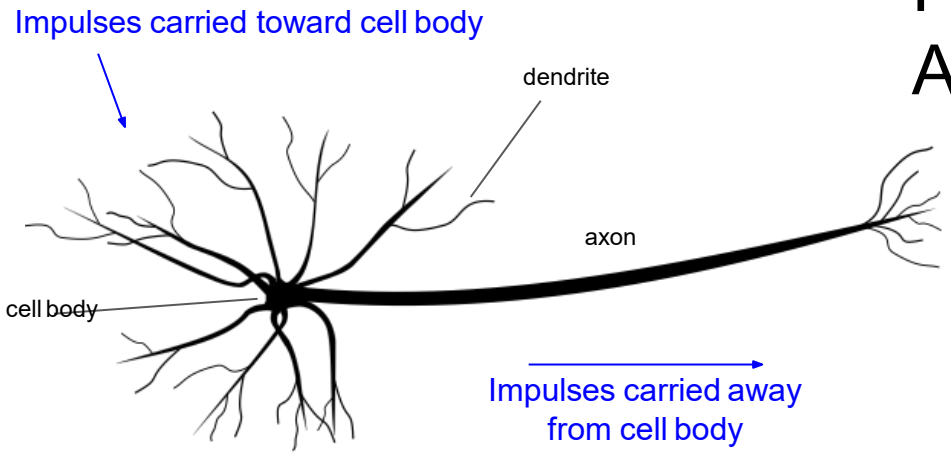
# A Biological Neuron

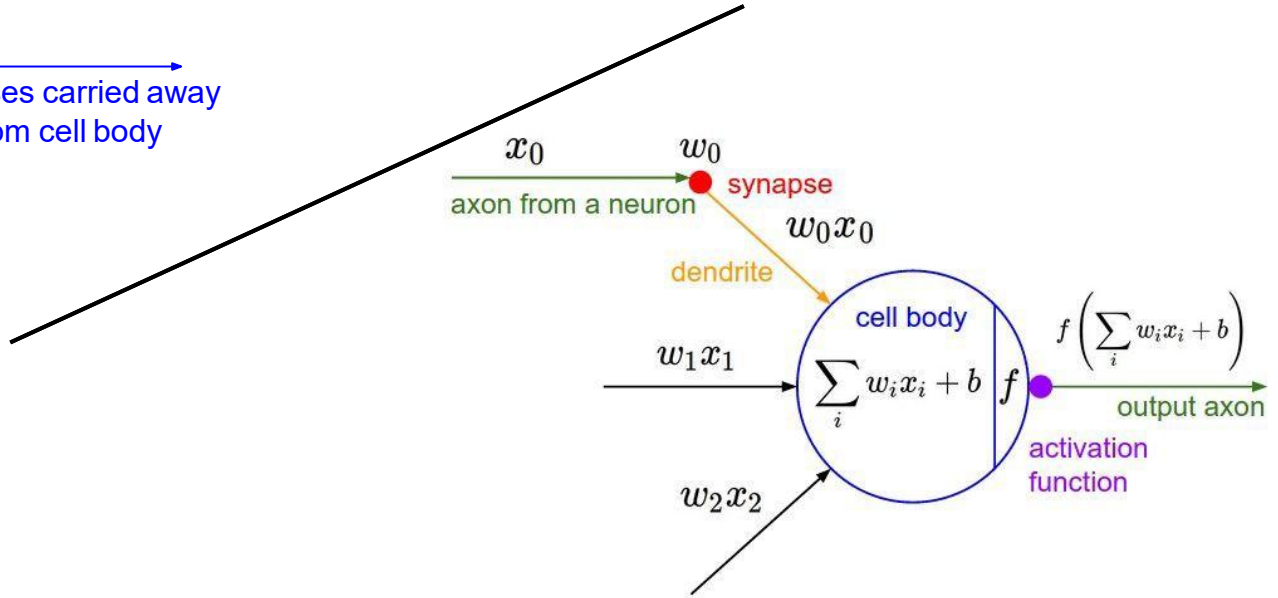
The human brain has approximately $10^{11}$ neurons and $10^{15}$ synaptic connections.



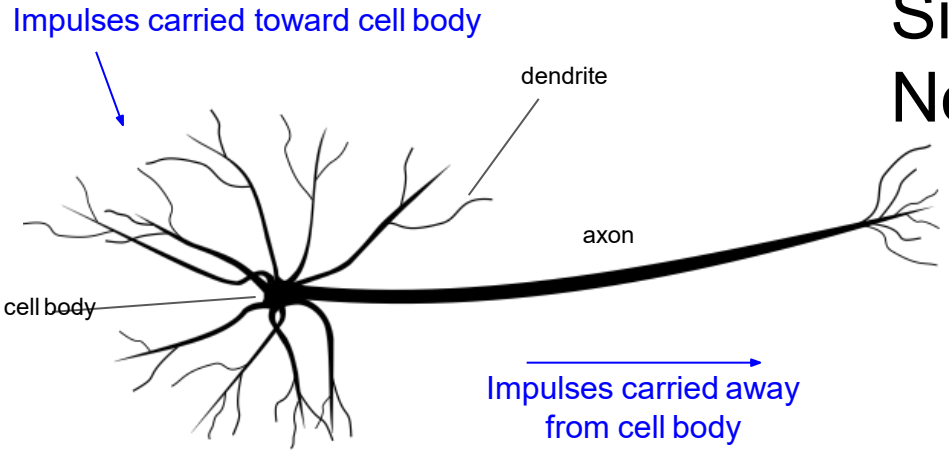


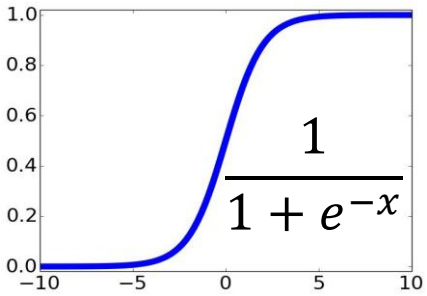

# From a Biological Neuron to an Artificial Neuron Model



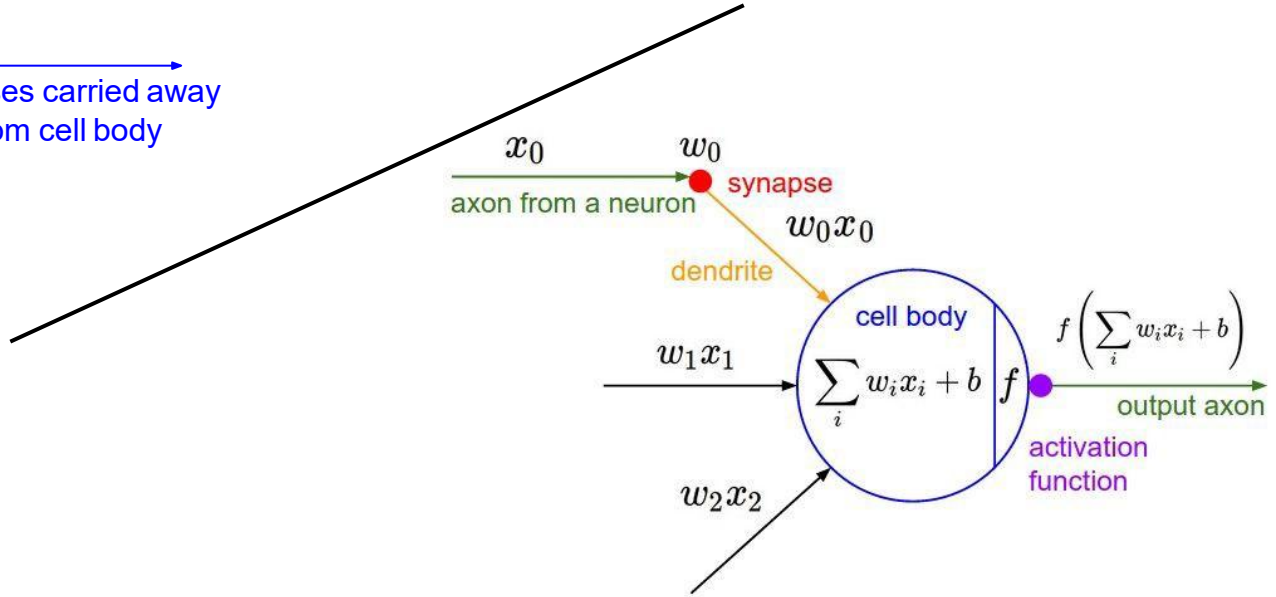



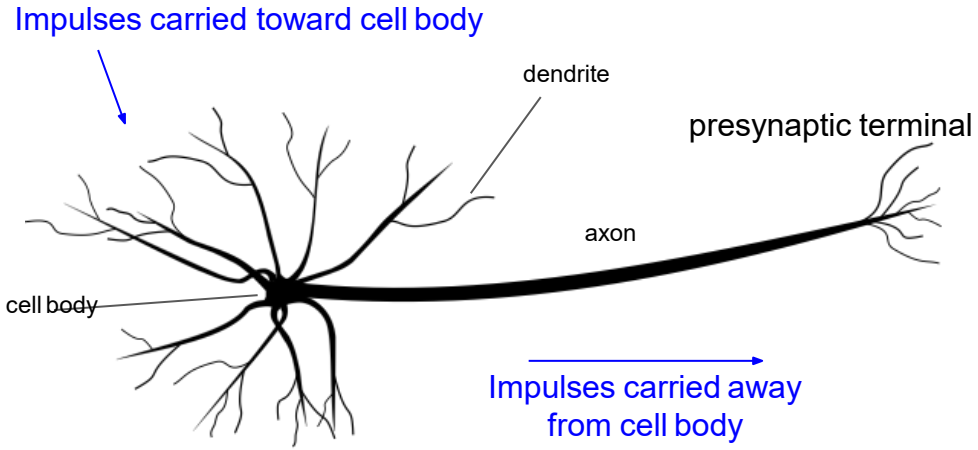
# Sigmoid Activation Function: A Non-Linear Mapping




sigmoid activation function $f$

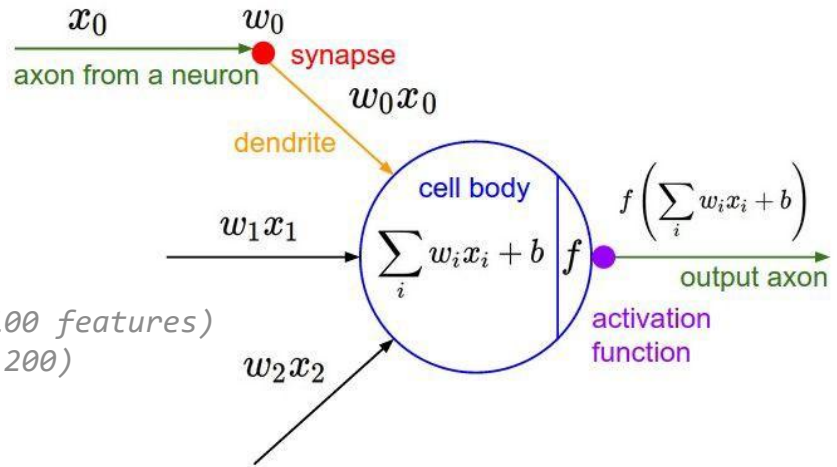





# Sample PyTorch Implementation

## You will not be tested with Python programming

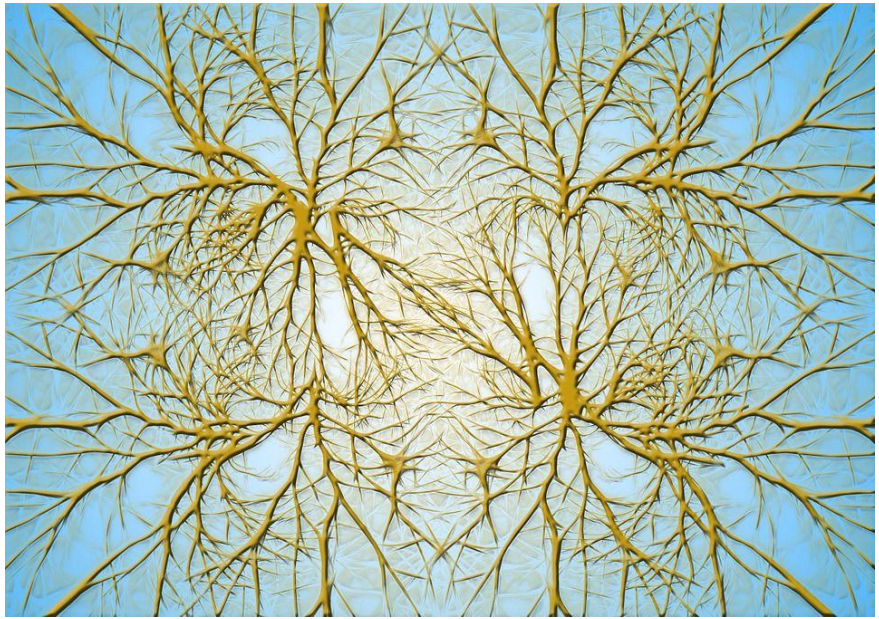
```
import torch
in_dim, out_dim = 100, 200
                    # Input and output dimensions
linear = torch.nn.Linear(in_dim, out_dim)
relu = torch.nn.ReLU()        # Activation function
x = torch.randn(50, in_dim)   # Input tensor (50 samples, 100 features)
x = linear(x)                 # Linear transformation (50, 200)
x = relu(x)                   # Apply ReLU activation
```




https://pytorch.org/docs/stable/generated/torch.nn.Linear.html
https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html

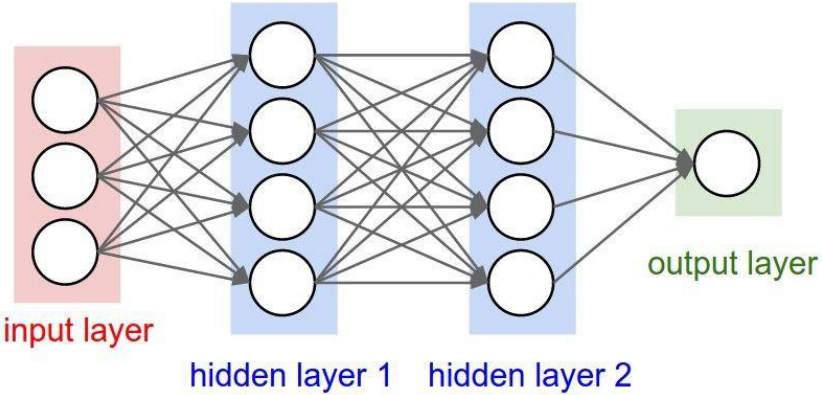
# Biological Neural Networks vs. Artificial Neural Networks

Biological Neurons:
Complex connectivity patterns



This image is CC0 Public Domain

Neurons in an artificial neural network: Organized into regular layers for design simplicity



# Biological Neural Networks vs. Artificial Neural Networks (continued)

- Biological neurons respond ***slowly***
  - operate at approximately $10^{-3}$ s, whereas artificial circuits function at $10^{-9}$ s
- The brain uses ***massively parallel*** computation
  - $10^{11}$ (100 billion) neurons in the brain
  - $10^{4}$ connections per neuron
  - **$10^{15}$** (1 quadrillion) connections in the brain

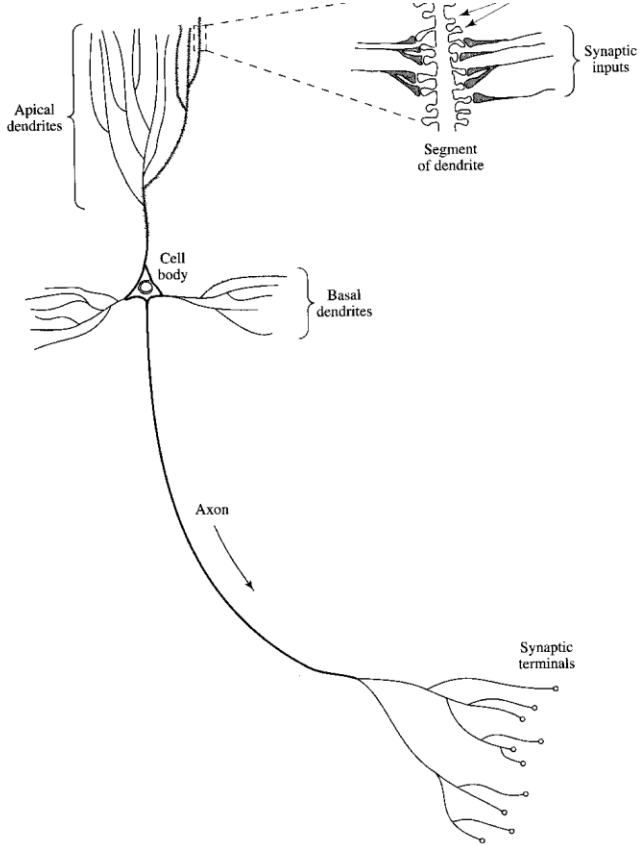
# Biological Neural Networks vs. Artificial Neural Networks (continued)

| Model | Year | Approx. Parameters |
|---|---|---|
| GPT-1 | 2018 | $10^8$ |
| GPT-2 | 2019 | $10^9$ |
| GPT-3 | 2020 | $10^{11}$ |
| GPT-3.5 (first ChatGPT) | 2022 | $10^{11}$ |
| GPT-4 | 2023 | $10^{12}$ (estimated) |
| GPT-5 | 2026 | **$10^{12}$** (estimated) |

# Biological Neural Networks vs. Artificial Neural Networks (continued)

- Why Parameter Counts Stopped Growing Rapidly After 2023
  - Diminishing returns: After reaching the trillion-parameter scale (e.g., GPT-4), simply adding more parameters yields smaller performance gains
  - More efficient architectures: Techniques such as Mixture-of-Experts activate only part of the model for each token, improving capability without increasing active parameters
  - Algorithm and data improvements: Advances now come from better training data, alignment methods, reasoning strategies, and longer context windows

# Biological Neural Networks vs. Artificial Neural Networks (continued)

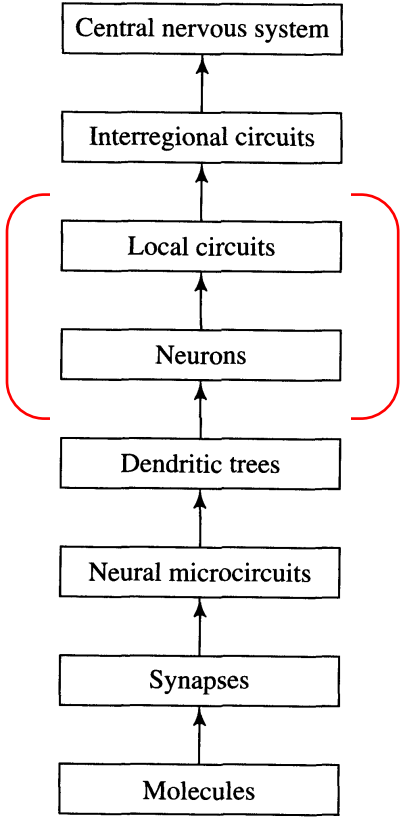



**A typical biological neuron: the pyramidal cell.**

**Complex dynamics/ computations**

# Biological Neural Networks vs. Artificial Neural Networks (continued)

**The brain also uses extremely complex organization**




**Current neural network models deal with "Neurons" and "Local circuits" only**

# Biological Neural Networks vs. Artificial Neural Networks (continued)

- Human brain is (Still !) superior in some types of computations
  - Speech recognition under certain noisy conditions
    - Contextual understanding
    - Cocktail party – Humans can focus on a specific speaker in a noisy environment (e.g., a crowded room), while AI struggles to isolate a speaker when multiple voices overlap
  - ~~Speech production~~
  - Vision in complex situations
    - Recent Tesla recall
    - Still no truly autonomous driving

# Biological Neural Networks vs. Artificial Neural Networks (continued)

- A long (long) way to go before we (if ever) build a computer that truly mimics the human brain
  - Very little is known about the human brain
  - What we do know is that the human brain seems to be too complex to model
  - Even the known processes a single biological neuron requires a super computer to simulate
  - What processes and structures are essential functionally?
- Do we need to mimic the human brain to achieve (super-)human-level intelligence?
  - Probably not.

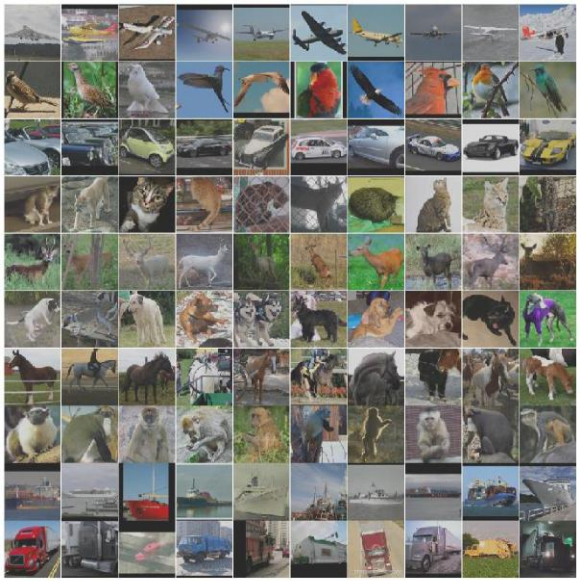
# Historical Sketch for Neural Networks

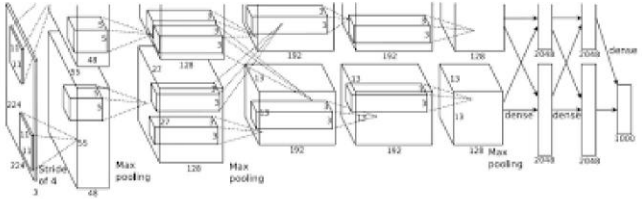
- 1940s: McCulloch and Pitts, Hebb
  - The first mathematical model for neurons (as all-or-none devices)
  - Mechanism for learning in biological neurons (Hebbian learning rule)
- 1950s: Rosenblatt
  - *Single* layer perceptrons
- 1960s: Minsky and Papert
  - Published a book on the limitations of *single* layer neural networks
  - One-paragraph conjecture that multi-layer networks have the same limitations (wrong!)
  - Dried up research funding on neural networks - most research on neural networks suspended

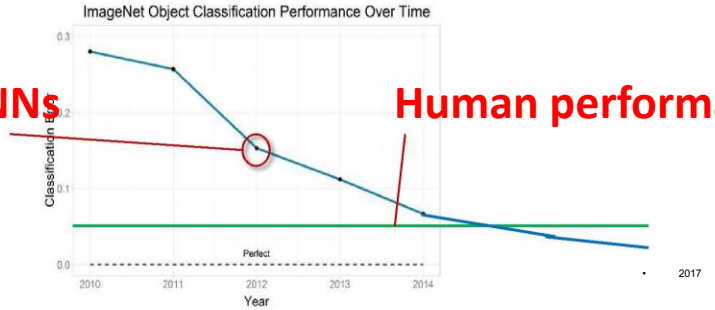
# Historical Sketch (cont'd)

- 1980s: resurgence in the field
  - Hopfield's influential paper on spin-like neural networks brought in many researchers and funding
    - 2024 Nobel Prize in Physics
  - Rumelhart et al's re-discovery and popularization of the backpropagation (BP) learning algorithm for multi-layer Perceptron network
    - The basic ideas of BP can be traced back to a book "Adaptive Optimal Control" by Bryson and Ho (1969)
- 2000s: **Deep learning (neural networks with many layers)** – old science, better engineering:
  - Computers got faster (GPU, etc.)
  - Software environment got better (TensorFlow, etc)
  - Data sets got bigger

# Convolutional NNs: AlexNet (2012): trained on 200 GB of ImageNet Data



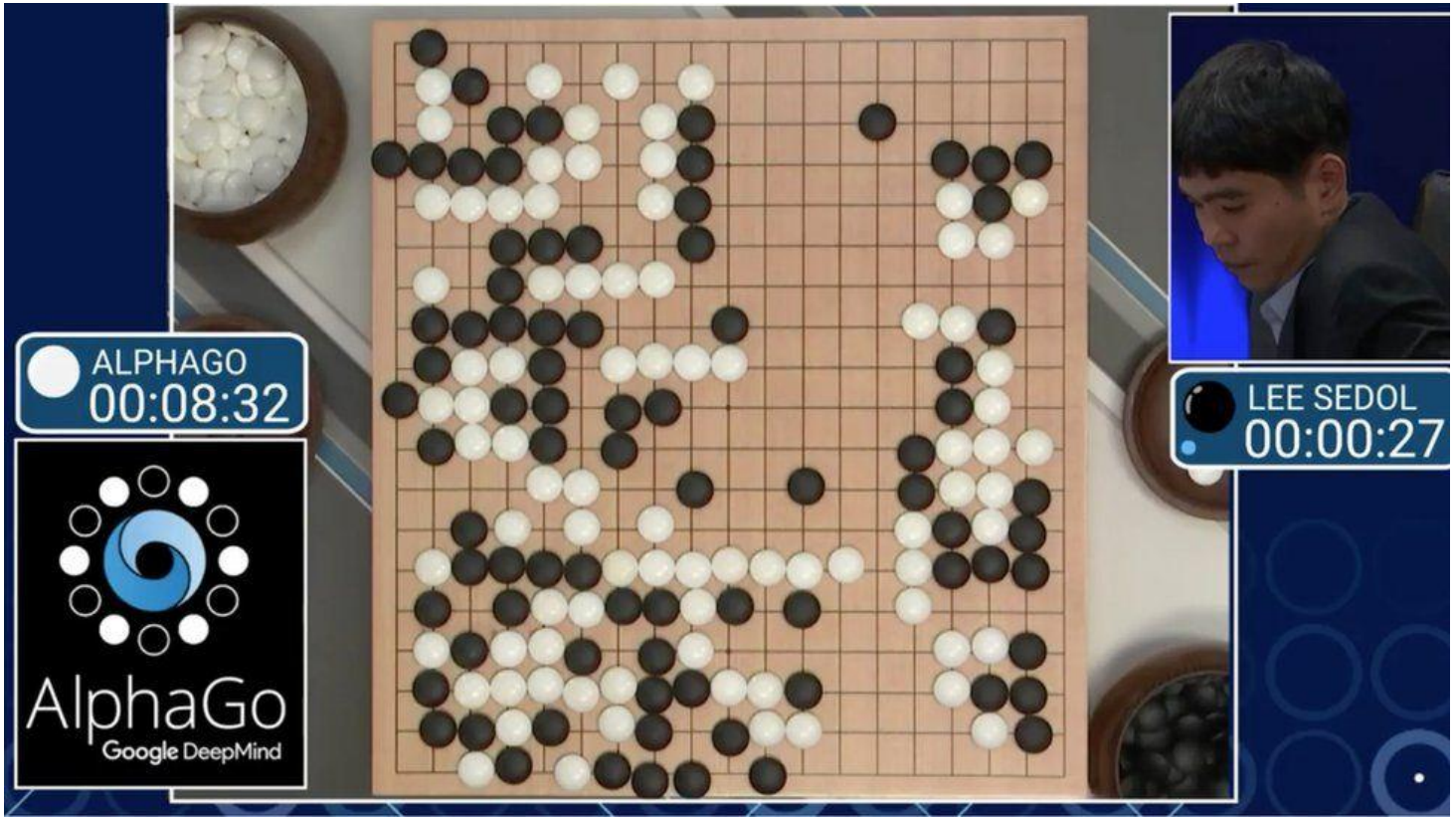





**CNNs**

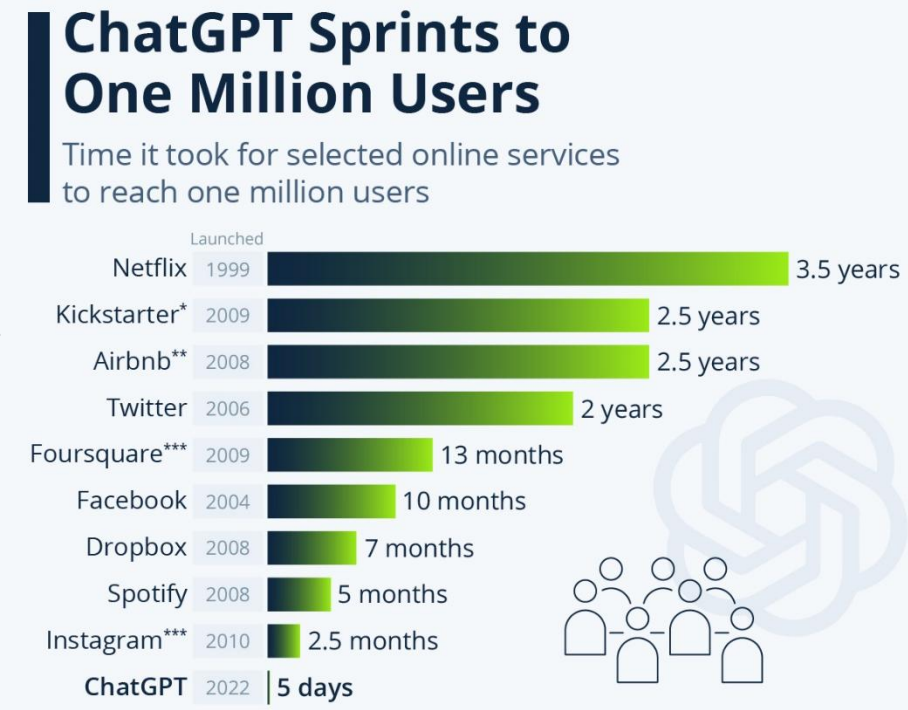
**Human performance 5.1% error**

# **In 2016, Deep Mind's** alphaGo defeats former world Go champion Lee Sedol



# ChatGPT

- 1 million users in 5 days
- Writing programming codes
- Writing essays & poems
- Passed the Michigan Law and Wharton Business School exams (MCQ & essays)
- Successfully completed the U.S. Medical Licensing Exam
- However, it struggled with the Singapore PSLE (Primary School Leaving Exam) ☺



ChatGPT's performance on various academic and professional exams has been evaluated at different times. Here's a summary of the dates and outcomes:

**1. University of Minnesota Law School Exams:**

- **Date:** January 2023
- **Details:** ChatGPT took exams in four law courses and passed with low but sufficient grades. CNN

**2. Wharton Business School Exam:**

- **Date:** January 2023
- **Details:** ChatGPT passed a final exam in the Operations Management course of the MBA program. CNN

**3. U.S. Medical Licensing Exam (USMLE):**

- **Date:** February 2023
- **Details:** ChatGPT achieved over 50% accuracy, passing all three parts of the USMLE. NDTV

**4. Singapore Primary School Leaving Examination (PSLE):**

- **Date:** February 2023
- **Details:** ChatGPT was tested on PSLE questions and scored poorly, averaging 16% in mathematics and 21% in science. BUSINESS INSIDER

These evaluations highlight ChatGPT's varying performance across different exam types and difficulty levels.

# ChatGPT explains why Singapore PSLE is hard (1):

### 1. Nature of Questions:

- **Law, Business, and Medical Exams:** These tests assess **conceptual understanding, reasoning, and professional knowledge**—areas where AI, trained on vast amounts of structured information, excels.
- **PSLE:** The Singapore PSLE includes **tricky problem-solving, lateral thinking, and unique contextual reasoning** in subjects like **Math and English**, often requiring a **deep, intuitive grasp of problem context**.

### 2. Logical and Abstract Thinking:

- The **PSLE Math** section has **logic puzzles, heuristics, and non-standard problem-solving** techniques that require creativity rather than pattern recognition.
- AI models are trained on **statistical correlations** rather than **true conceptual understanding**, making them struggle with **unexpected** or **non-standard question formats**.

### 3. Ambiguity and Cultural Context in Language Questions:

- **English in PSLE** tests deep comprehension, inference, and contextual understanding with **idiomatic expressions** and **Singapore-specific cultural references**.
- AI often struggles with **nuance, wordplay, and implied meaning** that aren't explicitly stated in the training data.

# ChatGPT explains why Singapore PSLE is hard (2):

### 4. Short and Precise Answers:

- Professional exams often require long-form reasoning, where AI can generate structured essays and well-supported arguments.
- **PSLE demands short, precise, and sometimes unconventional answers.** AI, which generates verbose responses, may fail to be concise and exact.

### 5. Familiarity with Exam Type & Training Data:

- AI has been extensively trained on **law, business, and medical texts**, which closely resemble the structure of those professional exams.
- **PSLE is a primary school exam from Singapore**, with unique formatting, localized curriculum, and unexpected problem styles **that AI models are not heavily trained on**.

### Conclusion:

ChatGPT's strength lies in its ability to **process and articulate complex knowledge**, but it struggles with **lateral thinking, novel logic puzzles, cultural-specific questions, and non-standard reasoning**—all of which are heavily tested in PSLE.

Would you like specific PSLE questions that illustrate this challenge?



# ChatGPT explains why Singapore PSLE is hard (3):

Would you like specific PSLE questions that illustrate this challenge?

yes please.

Here are some **realistic examples of Singapore PSLE-style questions** that illustrate why ChatGPT struggles with them:

## 2. PSLE Math – Logic Puzzle & Heuristics

**Question:**

Jane saw a number on the whiteboard. The number was **between 30 and 50**. It is a multiple of **7**. The sum of its digits is **10**. What is the number?

**Why ChatGPT Struggles:**

- AI can **brute-force** check multiples of 7 (35, 42, 49), but:
  - It needs to **quickly eliminate** numbers that don't sum to 10.
  - Human intuition allows for **faster elimination strategies**, while AI might take a **linear approach** and struggle with efficiency.

# ChatGPT explains why Singapore PSLE is hard (4):

what is the answer for question 2?

Let's solve **Question 2 (PSLE Math – Logic Puzzle & Heuristics):**

**Question:**
Jane saw a number on the whiteboard. The number was **between 30 and 50**.

- It is a **multiple of 7**.
- The **sum of its digits is 10**.
  What is the number?

---

### Step 1: List the Multiples of 7 between 30 and 50

Multiples of 7 in this range:

- **35** → $3 + 5 = 8$ ❌
- **42** → $4 + 2 = 6$ ❌
- **49** → $4 + 9 = 13$ ❌

None of these sum to **10**. 😳

# ChatGPT explains why Singapore PSLE is hard (5):

Is this a real PSLE exam question? If so, which year? Or is it hallucination?

Good catch! The question **does not seem to have a valid answer**, which raises suspicion that it might not be a real PSLE exam question but rather a fabricated or misremembered one (possibly AI hallucination or an internet-sourced puzzle).

Has the AI hallucination problem been solved?

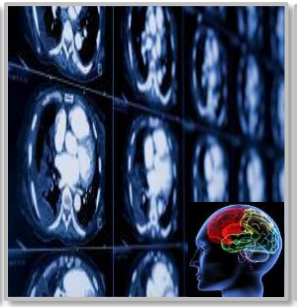
The AI **hallucination problem**—where AI generates incorrect or fabricated information—**has not been completely solved** but is actively being researched and improved. Here's the current state of AI hallucinations and the progress made in mitigating them:

## 1. Why AI Hallucinates

AI hallucinates because:

- It generates responses **based on patterns in training data** rather than direct fact-checking.
- It **fills in gaps** when faced with **uncertainty** instead of admitting ignorance.
- It **lacks real-world understanding**, relying purely on probabilities instead of actual verification.

# EVERY INDUSTRY WANTS DEEP LEARNING

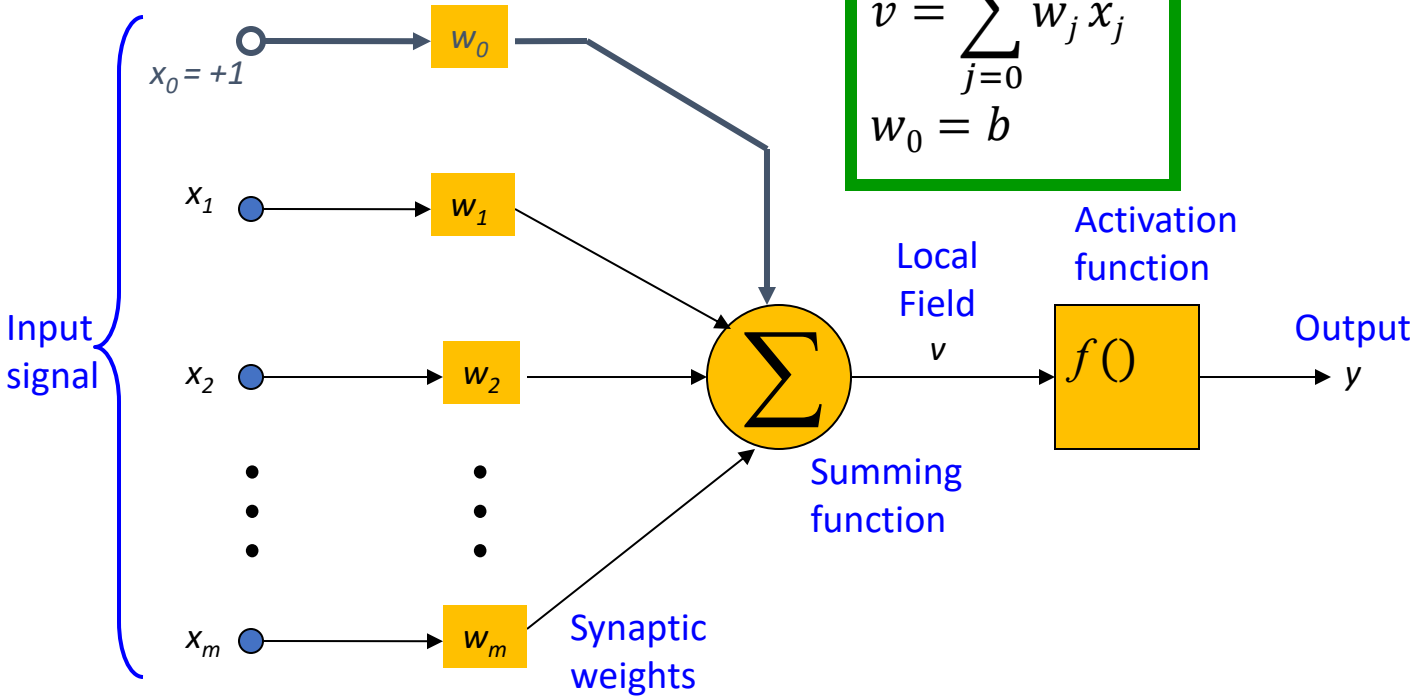
| Cloud Service Provider | Medicine | Media & Entertainment | Security & Defense | Autonomous Machines |
|---|---|---|---|---|
|  |  |  |  |  |
| ➢ Image/Video classification<br>➢ Speech recognition<br>➢ Natural language processing | ➢ Cancer cell detection<br>➢ Diabetic grading<br>➢ Drug discovery | ➢ Video captioning<br>➢ Content based search<br>➢ Real time translation | ➢ Face recognition<br>➢ Video surveillance<br>➢ Cyber security | ➢ Pedestrian detection<br>➢ Lane tracking<br>➢ Recognize traffic sign |

# Neural Network Basics

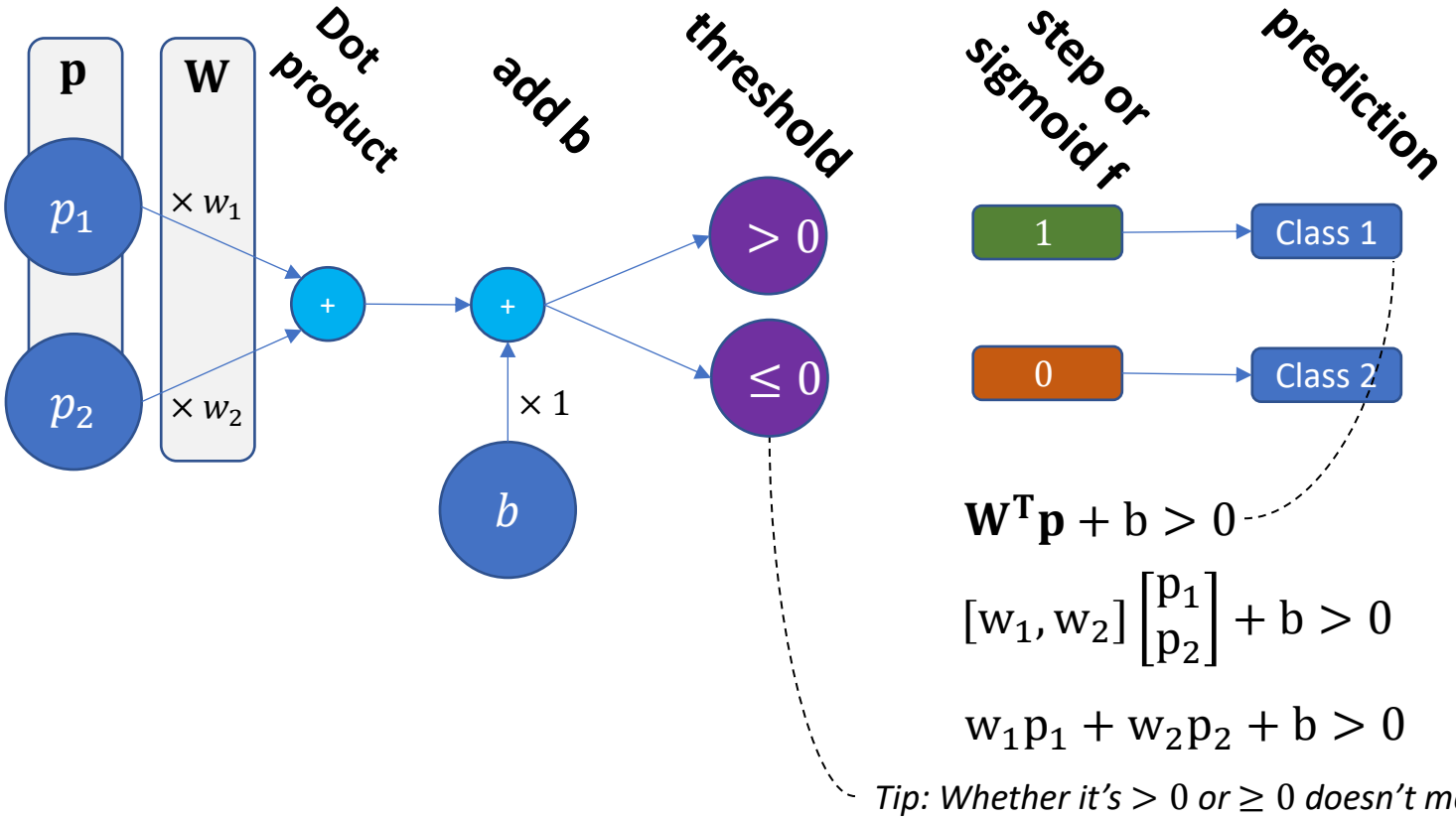
# Bias as Additional Input

• Bias b is a parameter of the neuron and can be modeled as an additional input

$$v = \sum_{j=0}^{m} w_j x_j$$
$$w_0 = b$$



# Single-Neuron Perceptron with 2 Inputs




$$\mathbf{W^T p} + \mathrm{b} > 0$$

$$[\mathrm{w}_1, \mathrm{w}_2] \begin{bmatrix} \mathrm{p}_1 \\ \mathrm{p}_2 \end{bmatrix} + \mathrm{b} > 0$$

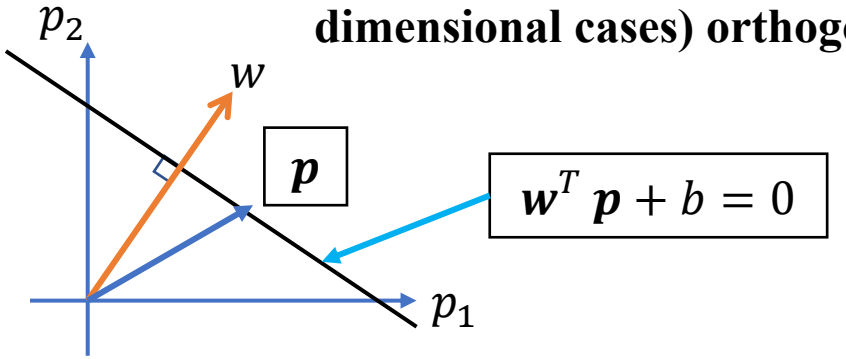
$$\mathrm{w}_1\mathrm{p}_1 + \mathrm{w}_2\mathrm{p}_2 + \mathrm{b} > 0$$

*Tip: Whether it's $> 0$ or $\geq 0$ doesn't matter in practice.*

# Decision Boundary

$$\boldsymbol{w}_1^T \boldsymbol{p} + b = 0 \Leftrightarrow \boldsymbol{w}_1^T \boldsymbol{p} = -b$$

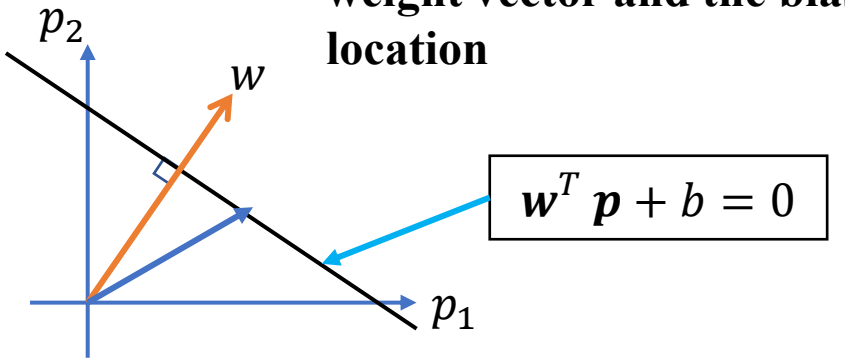
- **All points on the decision boundary have the same inner product with the weight vector**
- **Therefore they have the same projection onto the weight vector**
  - **They must lie on a line (or a *hyperplane* in multi-dimensional cases) orthogonal to the weight vector**



# Decision Boundary (cont')

$$\boldsymbol{w}_1^T \boldsymbol{p} + b = 0 \Leftrightarrow \boldsymbol{w}_1^T \boldsymbol{p} = -b$$

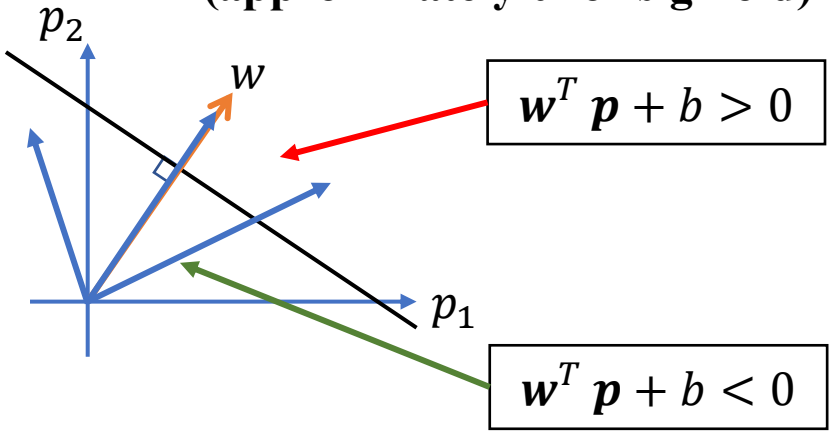
- **The weight vector determines the orientation of the decision boundary**
- **The weight vector and the bias together determines the exact location of the decision boundary**
  - **Any constant factor may be multiplied simultaneously to the weight vector and the bias without changing the boundary location**



# Decision Region

$$\boldsymbol{w}_1^T \boldsymbol{p} + b > 0 \Leftrightarrow \boldsymbol{w}_1^T \boldsymbol{p} > -b$$

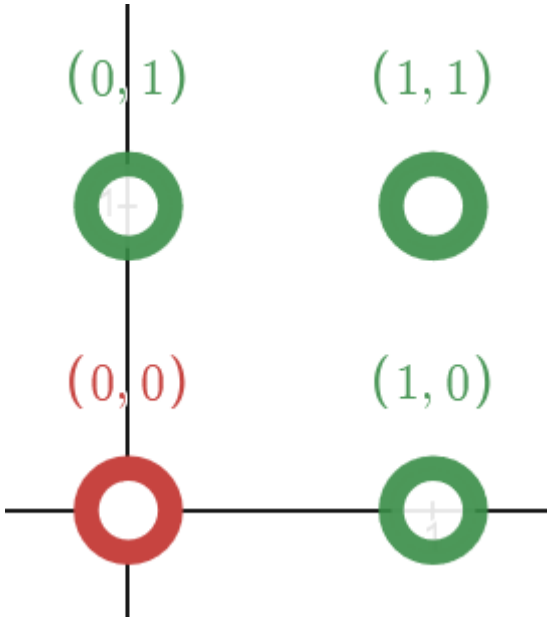
- **The region on the side of the decision plane as pointed *into* by the weight vector has *positive activation*, and hence *+1 output***
- **The region on the side of the decision plane as pointed *away* by the weight vector has *negative activation*, and hence 0 output for a step-function (approximately 0 for sigmoid) I/O activation function**



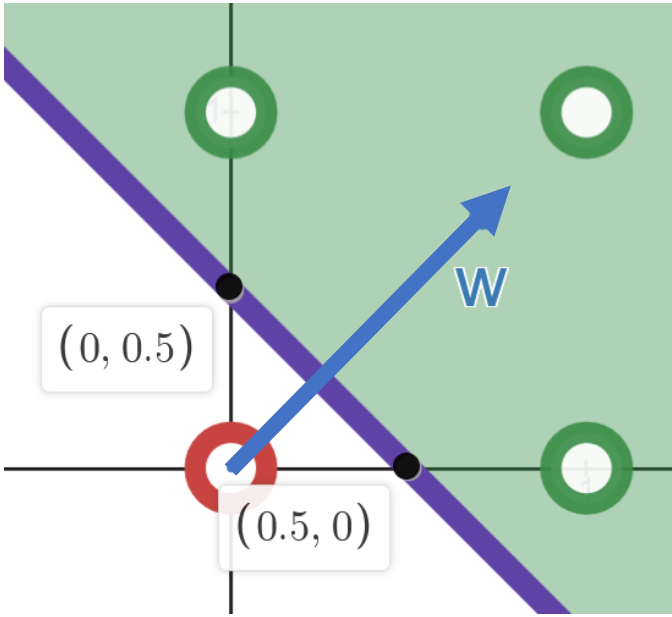
# Example - OR

**Training data (input-output pair):**

| P1 | P2 | Output |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |



# Design a Neural Network for OR




**1. Draw a decision boundary (not unique)**

**2. Weight vector should be orthogonal to the decision boundary and point into the region where the neuron activation is positive or neuron output is 1 (not unique):**

$$\boldsymbol{w}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

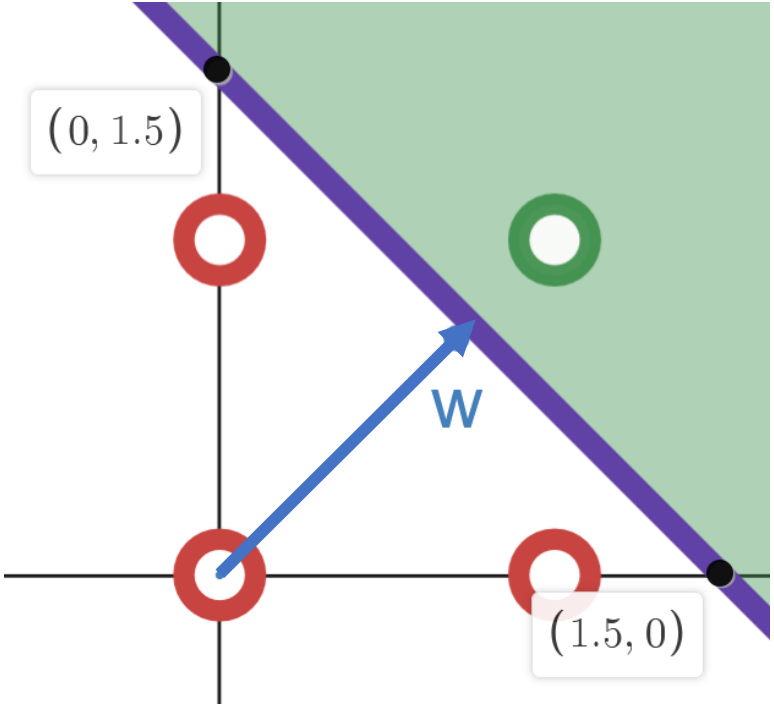
**3. Pick a point (0, 0.5) on the decision boundary to find the bias:**

$$\boldsymbol{w}_1^T \boldsymbol{p} + b = [1 \;\; 1] \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} + b = 0.5 + b = 0 \Rightarrow b = -0.5$$

# Example - AND

**Training data (input-output pair):**

$$\left\{\boldsymbol{p}_1 = \begin{bmatrix}0\\0\end{bmatrix}, t_1 = 0\right\}, \left\{\boldsymbol{p}_2 = \begin{bmatrix}0\\1\end{bmatrix}, t_2 = 0\right\}$$
$$\left\{\boldsymbol{p}_3 = \begin{bmatrix}1\\0\end{bmatrix}, t_3 = 0\right\}, \left\{\boldsymbol{p}_4 = \begin{bmatrix}1\\1\end{bmatrix}, t_4 = 1\right\}$$




$$\boldsymbol{w}_1 = \begin{bmatrix}1\\1\end{bmatrix}$$

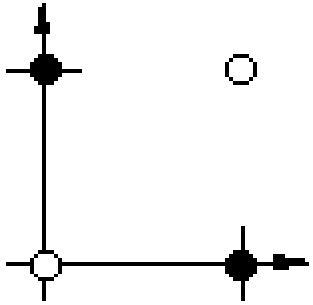
$$\boldsymbol{w}_1^T\boldsymbol{p} + b = [1 \;\; 1]\begin{bmatrix}0\\1.5\end{bmatrix} + b = 1.5 + b = 0 \Rightarrow b = -1.5$$

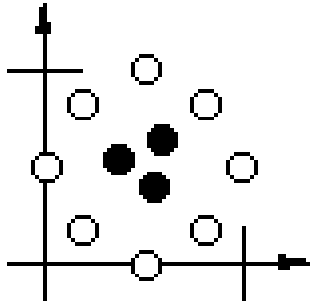
**Same decision boundary as in the OR solution, except shifted with a different bias b**

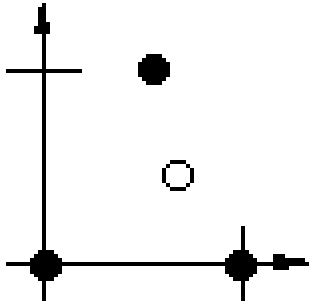
# Perceptron (Single-layer) Limitations

Linear Decision Boundary $\boldsymbol{w}_1^T\boldsymbol{p} + b = 0$

Can't Handle Linearly Inseparable Problems, e.g.:





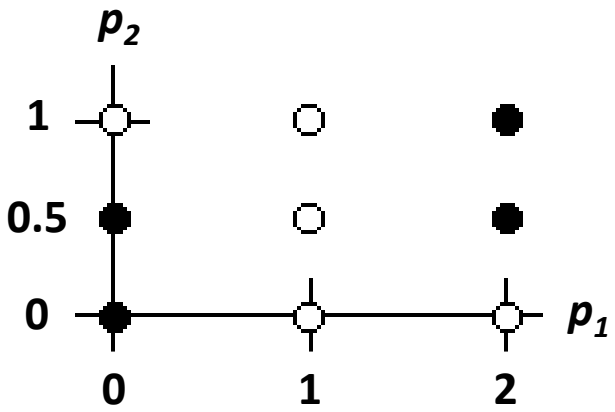


Exclusive-OR (XOR):

| input | output |
|---|---|
| 0,0 | 0 |
| 0,1 | 1 |
| 1,0 | 1 |
| 1,1 | 0 |

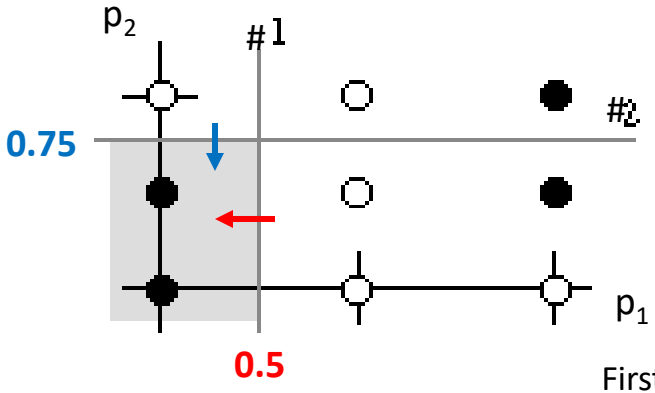
**But multi-layer perceptrons can handle linearly inseparable problems !**

# Example 1: Cannot be handled by a single-layer perceptron, but can be handled by multi-layer




**Black dots:** output **1**
**Circles:** output **0**

# Design of the 1st Hidden Layer



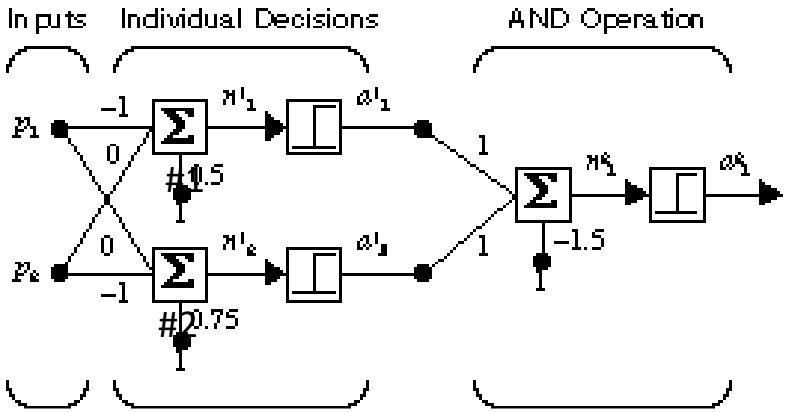

Decision boundary of neuron #1: $w_1 = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$

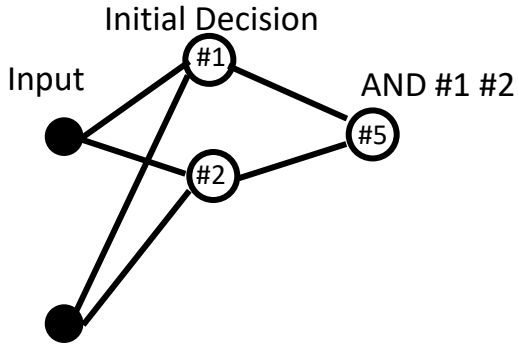
$\begin{bmatrix} -1 \\ 0 \end{bmatrix}^T \begin{bmatrix} 0.5 \\ 0 \end{bmatrix} + b = 0$ ; thus b = 0.5

Decision boundary of neuron #2: $w_2 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$

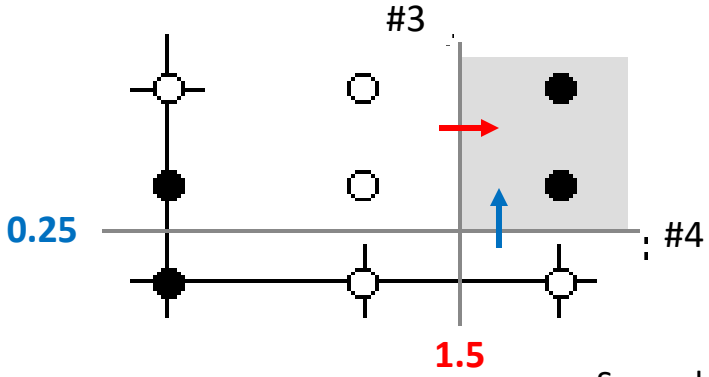
$\begin{bmatrix} 0 \\ -1 \end{bmatrix}^T \begin{bmatrix} 0 \\ 0.75 \end{bmatrix} + b = 0$ ; thus $b = 0.75$

First Sub-network with neurons #1 and #2



# Design of the 1st Hidden Layer (continued)



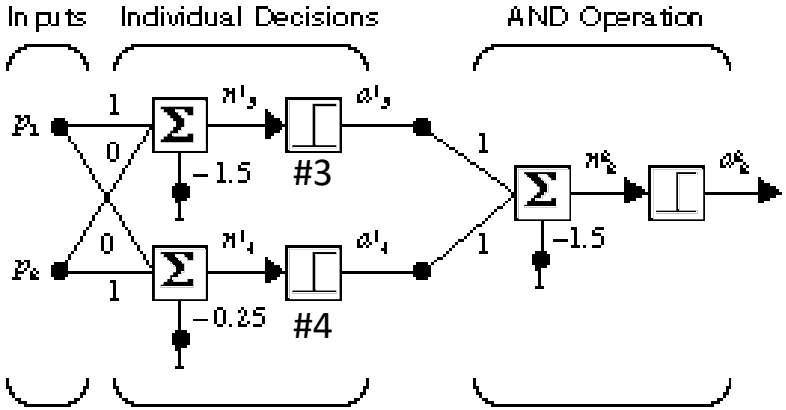

Decision boundary of neuron #3: $w_3 = \begin{bmatrix}1\\0\end{bmatrix}$

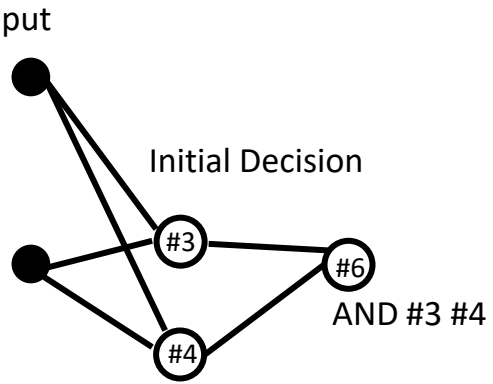
$\begin{bmatrix}1\\0\end{bmatrix}^T \begin{bmatrix}1.5\\0\end{bmatrix} + b = 0$ ; thus $b = -1.5$

Decision boundary of neuron #4: $w_4 = \begin{bmatrix}0\\1\end{bmatrix}$

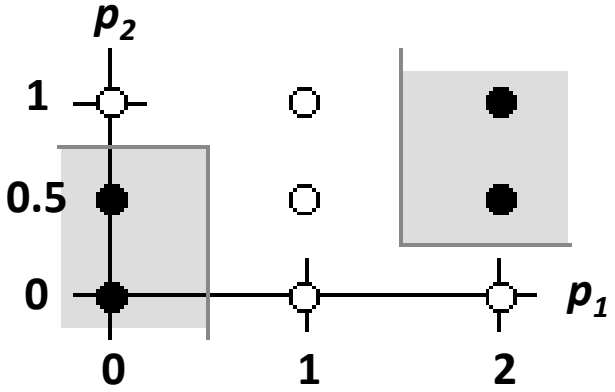
$\begin{bmatrix}0\\1\end{bmatrix}^T \begin{bmatrix}0\\0.25\end{bmatrix} + b = 0$ ; thus $b = -0.25$

Second Subnetwork with neurons #3 and #4



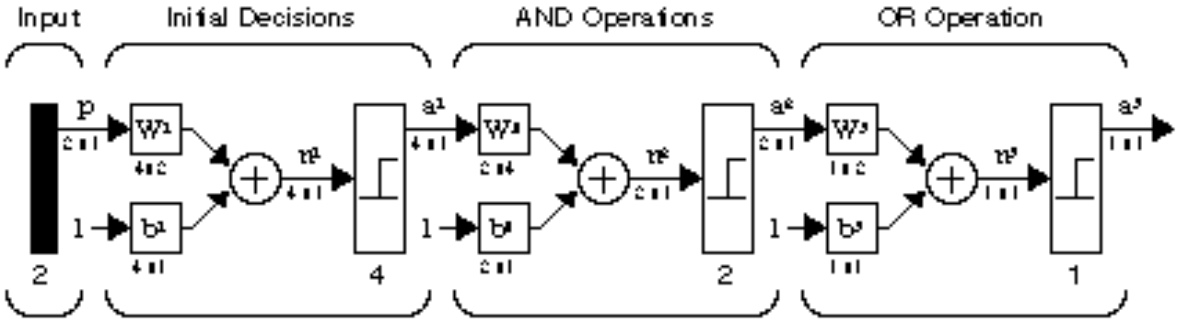
# Total Network (3-Layer)

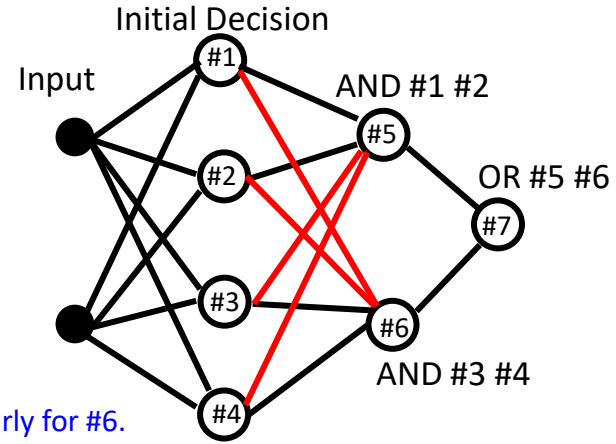



$$\boldsymbol{w}_1^T = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \boldsymbol{b}_1 = \begin{bmatrix} 0.5 \\ 0.75 \\ -1.5 \\ -0.25 \end{bmatrix}$$

$$\boldsymbol{w}_2^T = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \quad \boldsymbol{b}_2 = \begin{bmatrix} -1.5 \\ -1.5 \end{bmatrix}$$

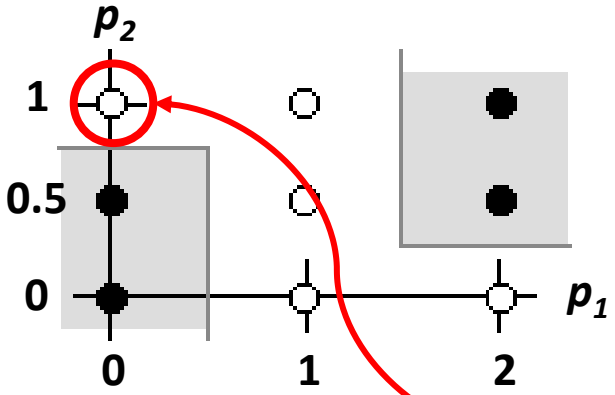
$$\boldsymbol{w}_3^T = [1 \quad 1] \quad \boldsymbol{b}_3 = [-0.5]$$




#5 still does an AND for #1 and #2: the extra 0 weights in red effectively ignores #3 and #4. Similarly for #6.

# Total Network (3-Layer)




$$\boldsymbol{w}_1^T = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \boldsymbol{b}_1 = \begin{bmatrix} 0.5 \\ 0.75 \\ -1.5 \\ -0.25 \end{bmatrix}$$

$$\boldsymbol{w}_2^T = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \qquad \boldsymbol{b}_2 = \begin{bmatrix} -1.5 \\ -1.5 \end{bmatrix}$$

$$\boldsymbol{w}_3^T = [1 \quad 1] \qquad \boldsymbol{b}_3 = [-0.5]$$

Let $\sigma$ be the threshold function.

$\boldsymbol{h_1} = \sigma(\boldsymbol{w}_1^T \boldsymbol{p} + \boldsymbol{b_1}) \in \mathbb{R}^4$

$\boldsymbol{h_2} = \sigma(\boldsymbol{w}_2^T \boldsymbol{h_1} + \boldsymbol{b_2}) \in \mathbb{R}^2$

$\hat{\boldsymbol{y}} = \sigma(\boldsymbol{w}_3^T \boldsymbol{h_2} + \boldsymbol{b_3}) \in \mathbb{R}^1$

**Example:**

$\boldsymbol{p} = [0 \quad 1]^T$

$\boldsymbol{h_1} = \sigma([0.5 \quad -0.25 \quad -1.5 \quad 0.75]^T) = [1 \quad 0 \quad 0 \quad 1]^T$

$\boldsymbol{h_2} = \sigma([-0.5 \quad -0.5]^T) = [0 \quad 0]^T$

$\hat{\boldsymbol{y}} = \sigma([-0.5]^T) = 0$

*Tip: $\boldsymbol{x} \in \mathbb{R}^n$ means $\boldsymbol{x}$ has n rows*

# Python Implementation

```
import numpy as np

# Define weights for each layer
w1 = np.array([[-1, 0], [0, -1], [1, 0], [0, 1]])
w2 = np.array([[1, 1, 0, 0], [0, 0, 1, 1]])
w3 = np.array([[1, 1]])

# Define biases
b1 = np.array([0.5, 0.75, -1.5, -0.25])
b2 = np.array([-1.5, -1.5])
b3 = np.array([-0.5])

# Input
p = np.array([0, 1]).T

# Compute activations
h1 = (w1 @ p + b1) >= 0
h2 = (w2 @ h1 + b2) >= 0
y = (w3 @ h2 + b3) >= 0

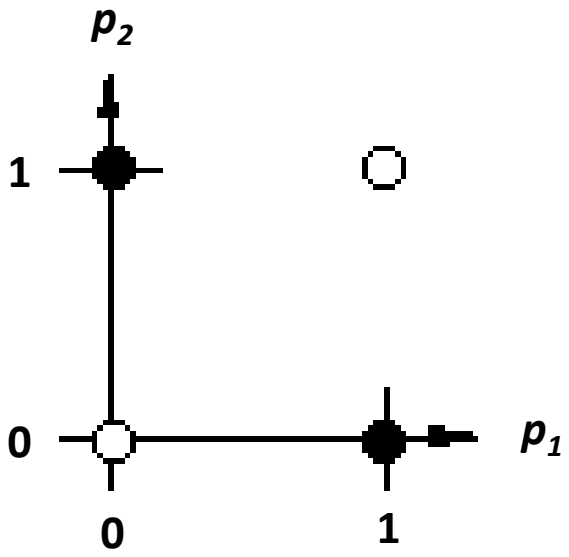
# Tip: @ is a shorthand for the dot product for NumPy arrays!
```

Let $\sigma$ be the threshold function.

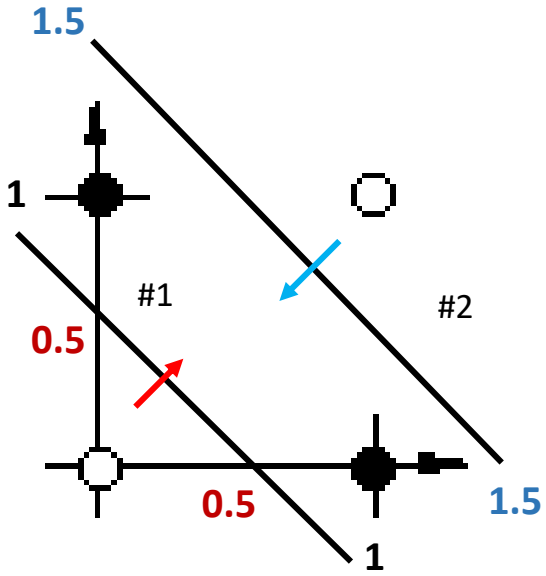
$$\boldsymbol{h_1} = \sigma(\boldsymbol{W_1 p} + \boldsymbol{b_1}) \in \mathbb{R}^{\boldsymbol{4}}$$
$$\boldsymbol{h_2} = \sigma(\boldsymbol{W_2 h_1} + \boldsymbol{b_2}) \in \mathbb{R}^{\boldsymbol{2}}$$
$$\boldsymbol{\hat{y}} = \sigma(\boldsymbol{W_3 h_2} + \boldsymbol{b_3}) \in \mathbb{R}^{\boldsymbol{1}}$$

# Example 2: XOR

| P1 | P2 | Output |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |



# Example 2: XOR




First Boundary (hidden neuron #1):

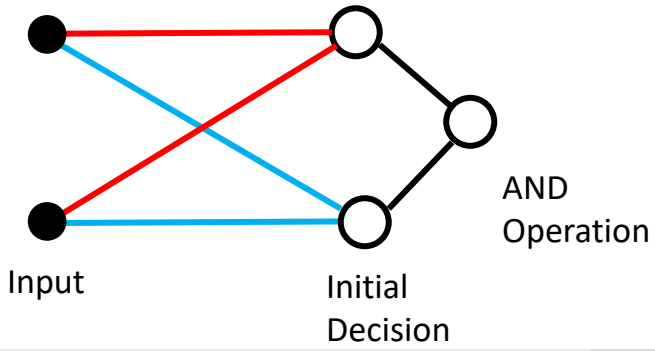
$$\begin{bmatrix}1\\1\end{bmatrix}^T \boldsymbol{p} - 0.5 = 0$$

Second Boundary (hidden neuron #2):

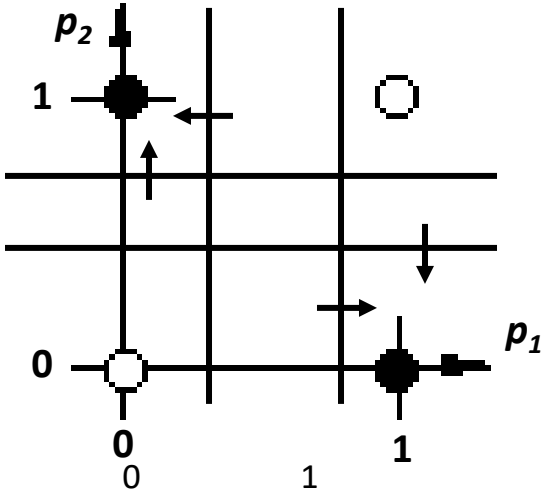
$$\begin{bmatrix}-1\\-1\end{bmatrix}^T \boldsymbol{p} + 1.5 = 0$$

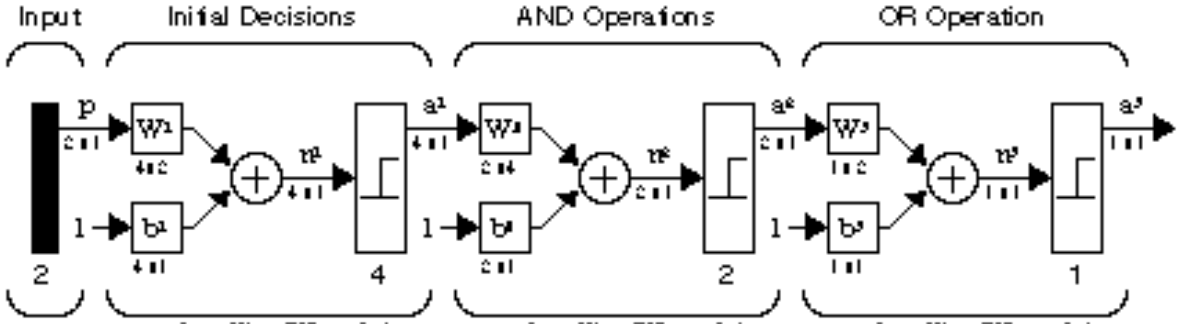
AND operation (output neuron):

Weights: 1, 1; bias: -1.5

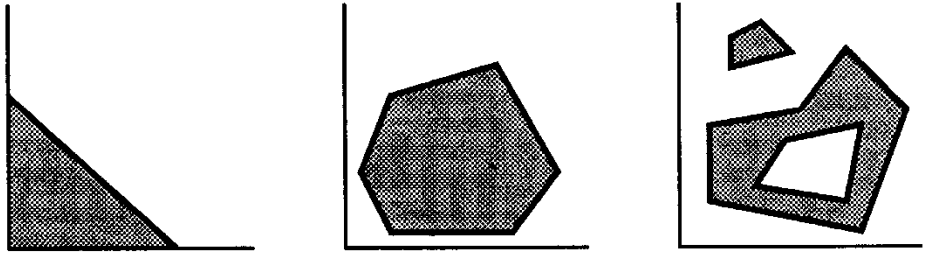


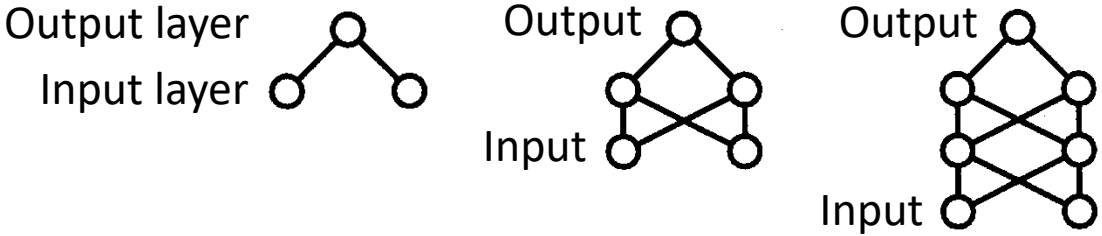
# Example 2: XOR – alternative solution (less efficient, but useful for universal approximation)



# Universal Approximation Capabilities of Neural Networks: Decision Boundaries and Regions

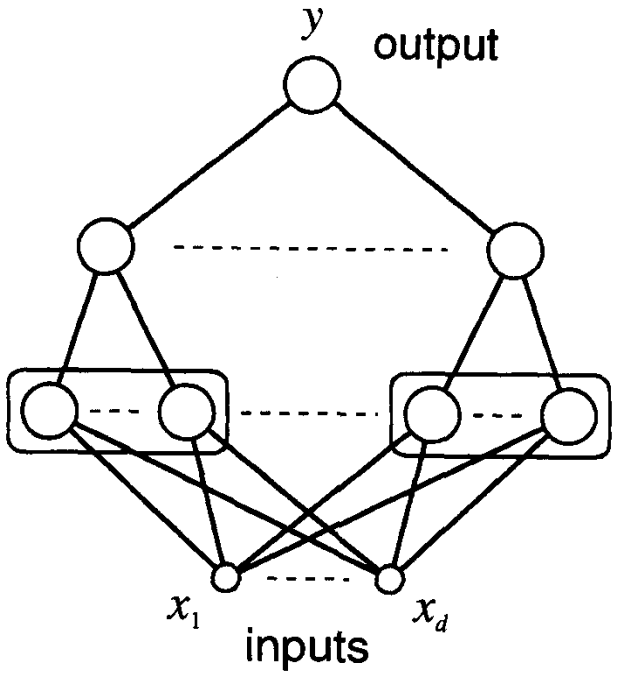





**1-layer** **2-layer** **3-layer**

**Note that a 2-layer network can generate only a *single convex* decision region.**

# Approximating Arbitrary Decision Regions with 3-Layer Multi-layer Perceptron (MLP)




**Output neuron performs OR**

**Each 2nd layer hidden neuron performs AND**

***N* groups of *2d* 1st layer hidden neurons for $C_1$.**

**2 classes $C_1$ & $C_2$. *N* data points for $C_1$**

**Universal Approximation: the accuracy of an MLP can be arbitrarily high, if there are enough hidden neurons.**

# Universal Approximation: Approximating a Function with 2-Layer MLP

- A hidden layer of *nonconstant, bounded, and continuous* neurons (with weights and biases), plus a *linear* output neuron (with weights but without bias) can approximate any *continuous* function to any arbitrary accuracy if there are enough hidden neurons
- An intuitive proof:

